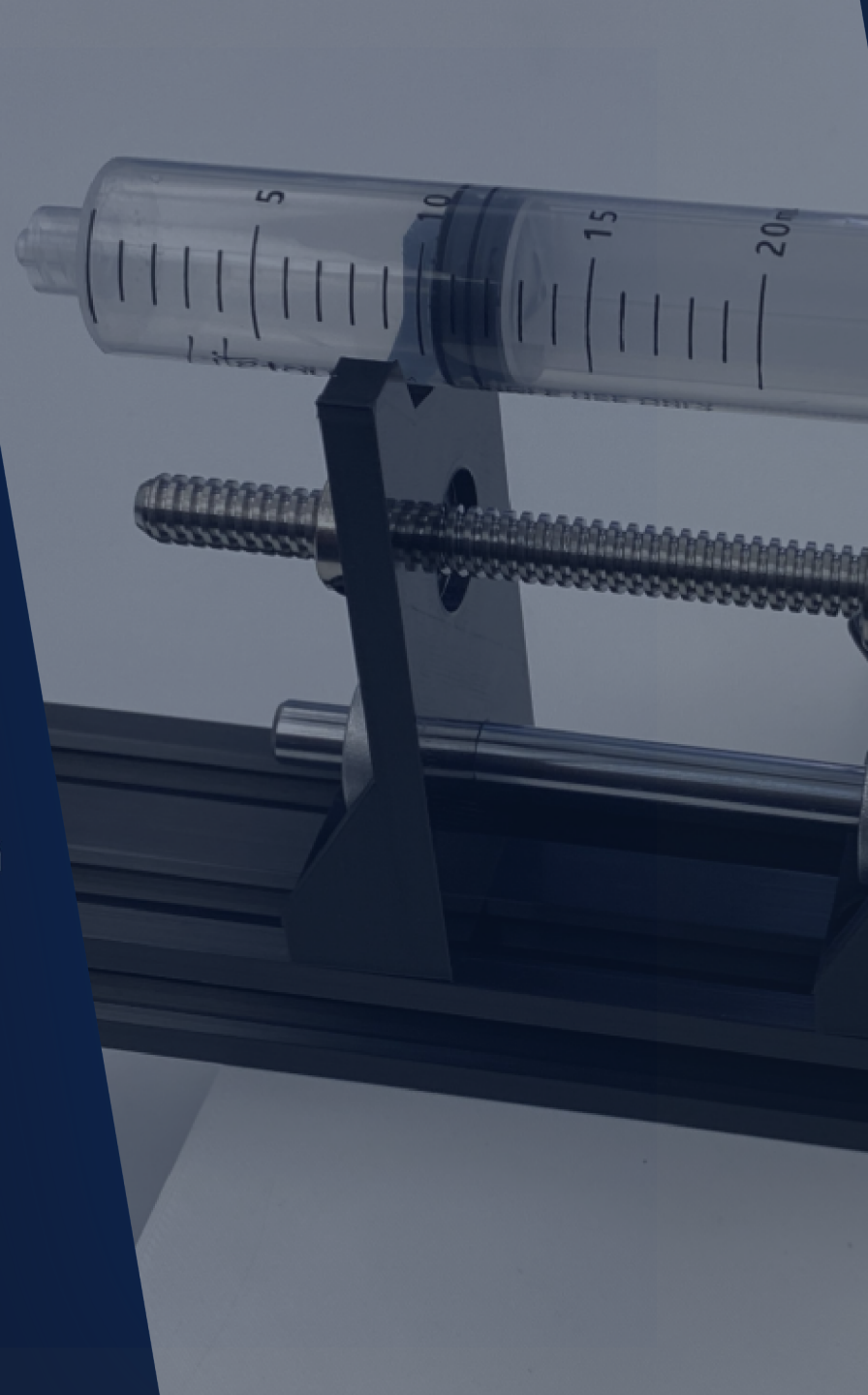
LABORATORY 4.0 CHALLENGE · AXIS 1 · IEEE TRSYP 3.0

LabPilot

A self-verifying syringe pump for automated sample preparation



The problem: a command is not proof

What can go wrong?

- Incorrect delivered volume
- Inconsistent repeated procedures
- Mechanical failures that go unnoticed
- Incomplete records of what actually happened

Manual preparation

Repetitive work can vary between operators.

Conventional automation

A motor command does not guarantee successful delivery.

The missing step

Verify the physical result before accepting the sample.

How do we know the requested quantity was actually delivered?

Witnesses

CROSS-CHECK → VERDICT

Required checks pass: ACCEPTED |
Disagreement: named fault

01 MOTION

Arduino step count
What movement was commanded?

02 VISION

Camera + OpenCV
Is the container aligned? Did the plunger move?

Any disagreement stops the sample with named fault, and every result is logges

Architecture

RASPBERRY PI · INTELLIGENCE

Protocol execution
OpenCV vision + decision logic
GUI, timestamped logs, run summary

ARDUINO UNO + CNC SHIELD · CONTROL

Stepper and lead-screw control
Load-cell and end-stop readings
Fast → slow → stop on measured mass

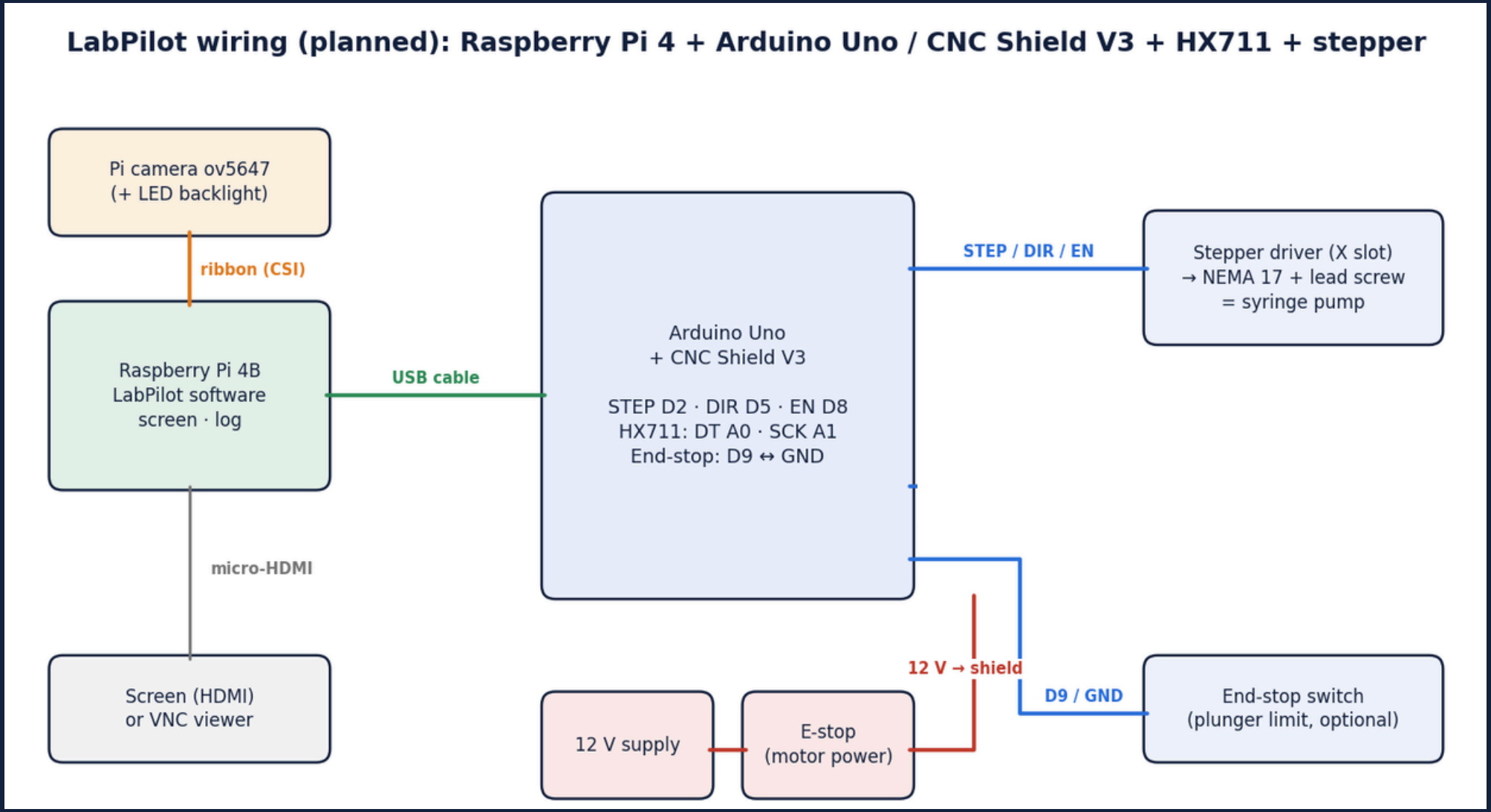
CAMERA + BACKLIGHT

Container position + plunger travel

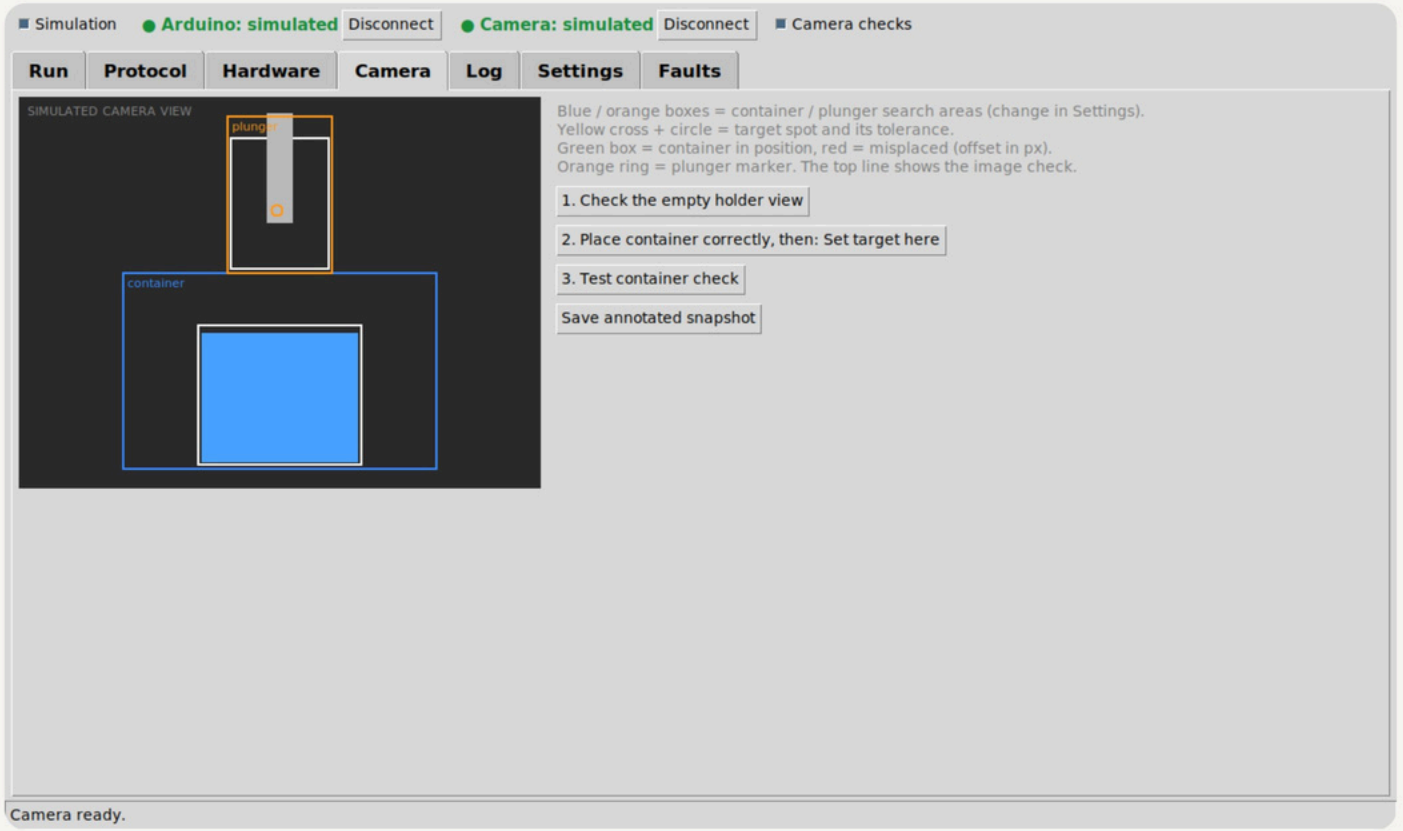
SYRINGE PUMP

Stepper motor + lead screw

System Cabling



What the camera checks



Container position

Dark silhouette on the backlight; offset from the target in pixels.

Image guard

Too dark, overexposed or blurry: the run continues on the scale alone.

Plunger tracking

A tape marker on the pump block; travel compared with the commanded volume.

Computer vision: check what is physically happening

IMAGE QUALITY

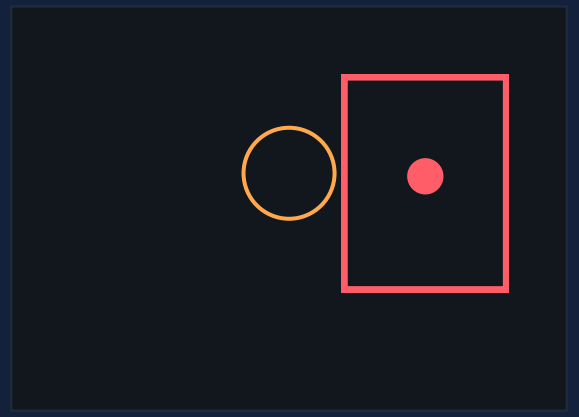
Brightness, white-pixel fraction, and sharpness



UNRELIABLE → ABSTAIN

CONTAINER POSITION

Backlit silhouette, bounding box, and offset from target



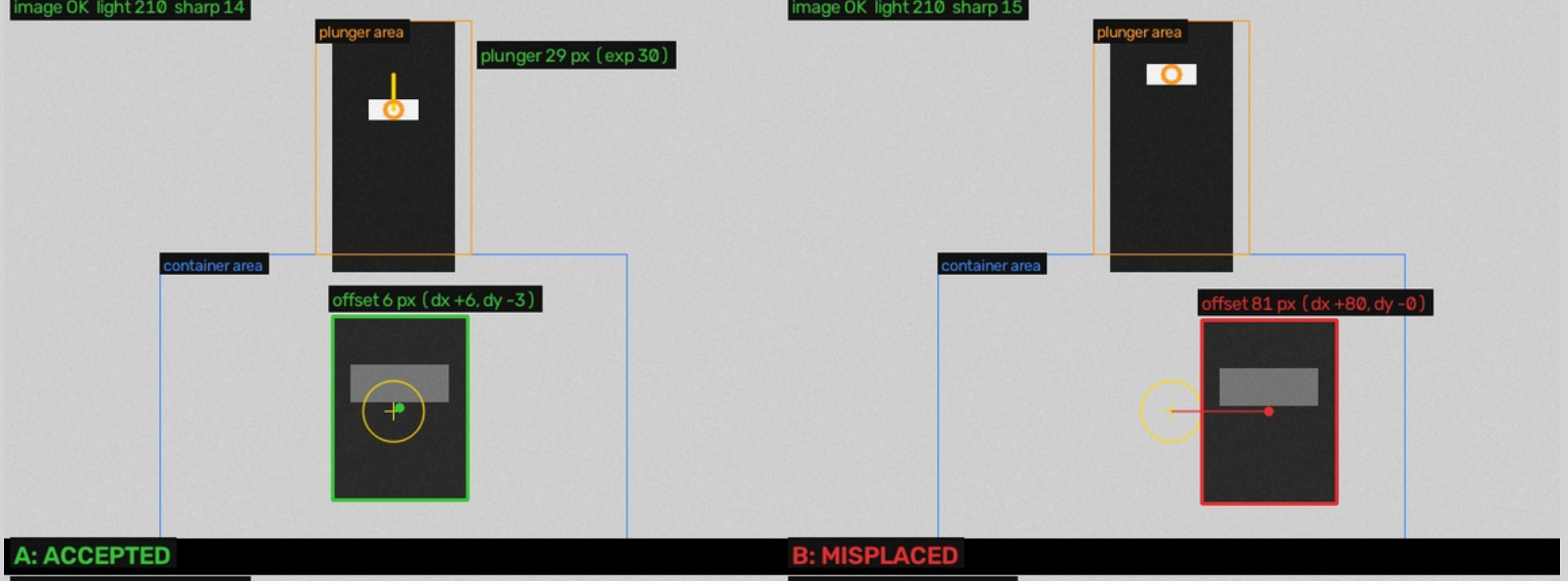
OFFSET IN PIXELS

PLUNGER TRACKING

Tape-marker centroid and observed travel



TRAVEL VS EXPECTED


image OK light 210 sharp 14
plunger area
plunger 29 px (exp 30)
container area
offset 6 px (dx +6, dy -3)
A: ACCEPTED
image OK light 210 sharp 15
plunger area
container area
offset 81 px (dx +80, dy -0)
B: MISPLACED
image OK light 210 sharp 14
plunger area
plunger 15 px (exp 39) SLIP
container area
offset 1 px (dx +0, dy -0)
D: PLUNGER_SLIP
image OK light 210 sharp 9
plunger area
container
no container silhouette
E: NO_CONTAINER

System Workflow

01

LOAD

Read protocol and target

02

CHECK

Container present and aligned

03

DOSE

Tare; fast then slow dosing

04

VERIFY

Wait for settled mass; compare

05

RECORD

Accept or reject; log the verdict

ACCEPTED

Required checks agree and final mass is within tolerance.

REJECTED / ABORTED

A failed check produces a named fault and a log entry.

A modular software and hardware stack

RASPBERRY PI · PYTHON

main.py — sample orchestration
vision.py / vision_core.py — image analysis
decision.py — verdict rules
link.py — serial link + simulated Arduino
gui.py — Tkinter interface
protocol.py / logger.py / config.py

ARDUINO · C++

Arduino Uno + CNC Shield V3
AccelStepper — motor control
EEPROM — saved scale calibration
USB serial — ASCII-line protocol
End-stop + physical motor emergency stop

09 / PROTOCOL-DRIVEN OPERATION

A digital protocol defines every sample

Protocol input

CSV / JSON file or form on the interface
Protocol ID, sample ID, target volume, tolerance

Example protocol

P-001 · Sample A · 5.0 mL · ±0.2 mL
P-001 · Sample B · 3.0 mL · ±0.2 mL
P-001 · Sample C · 5.0 mL · ±0.2 mL

Before execution

Validate target values and syringe capacity.

During execution

Process one sample at a time; show the next target.

After execution

Record the measurement, verdict and any fault.

The Pi and Arduino communication protocole

PI → ARDUINO

PING · TARE / UNTARE · MASS

PI → ARDUINO

DOSE <target_g> <max_steps> · STOP · CAL <known_g>

Arduino replies

OK PONG · OK TARE · MASS <g>
Calibration confirmation: OK CAL

Dosing stream

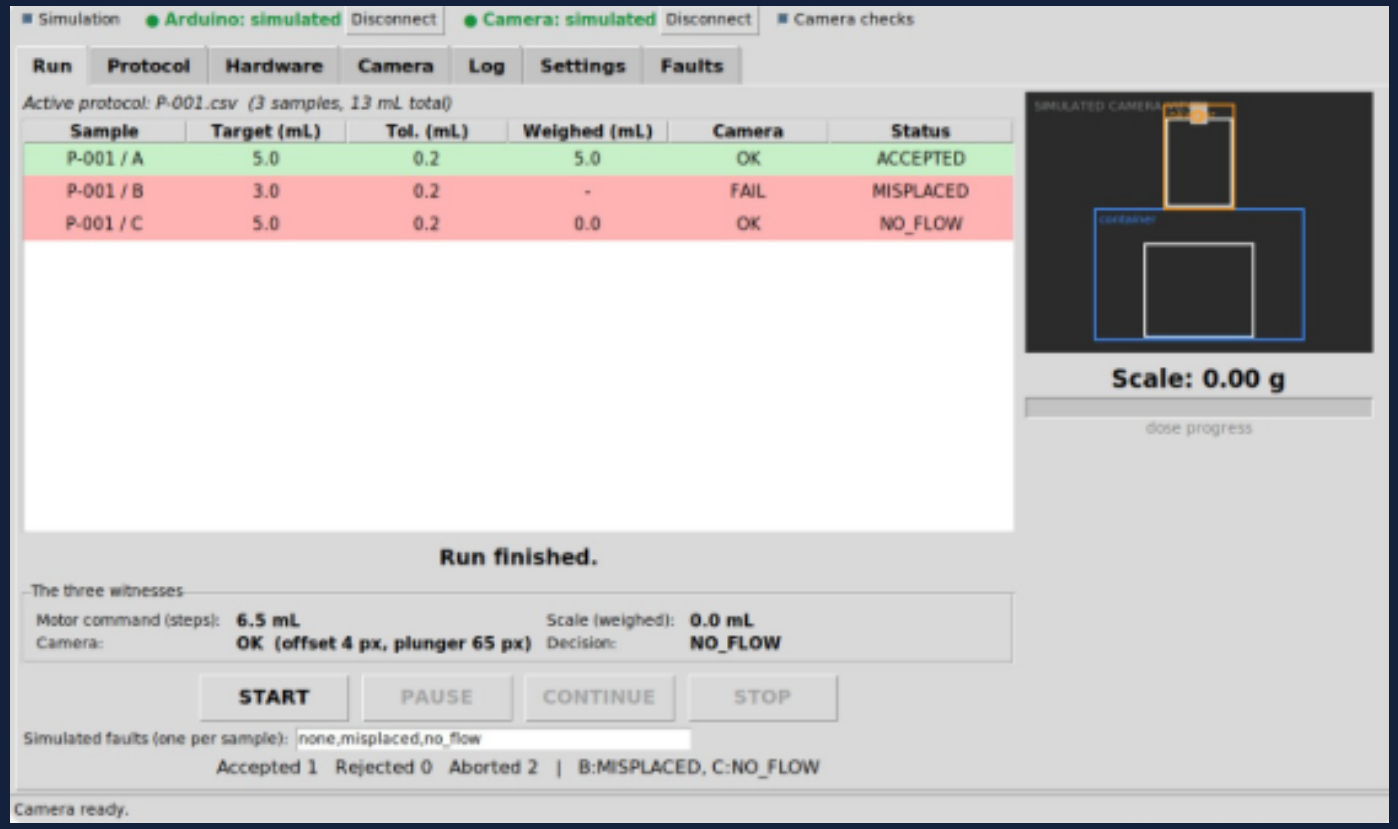
DATA <ms> <g> <steps> repeated during dosing
Final status: DONE, LIMIT or ENDSTOP

Transport: USB serial · 115200 baud · one ASCII line per message

Decision logic turns evidence into a verdict

MOTOR COMMAND	CAMERA	LOAD CELL	DECISION
Command: push	Plunger moves	Mass rises as expected	**Continue → ACCEPTED**
Command: push	Plunger moves	No mass gain	**NO_FLOW**
Command: push	Plunger still	No mass gain	**MOTOR_STALL**
Command: push	Plunger moves	Mass differs from target	**MISMATCH / OVERFILL**

Simulation



Sample1

Accepted : 0.5ml all witness agree

Sample 2

Misplaced : the camera does not detect the container in it’s place

Sample 3

NO_FLOW : the plunder is moving

Thank You

For Your Attention



Automation



Verification



Traceability